Section 4: Creational Design Patterns

This section is part of the 5-hour LLD course by Mukul Raina

- YouTube Course: https://www.youtube.com/watch?v=VSABVEBDhf0
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

SLIDE 41: Section Intro: Creational Design Patterns

SLIDE 42: Factory Method: Decoupling Object Creation from Usage

SLIDE 43: Wrong Approach: Switch Statement in Calling Code

SLIDE 43B: Right Approach: Factory Method Implementation

SLIDE 44: Worked Example: Notification Channel Creation

SLIDE 45: Abstract Factory: Families of Related Objects

SLIDE 45B: Worked Example: UI Component Factory for Themes

SLIDE 45C: Interview Practice: Factory Method and Abstract Factory

SLIDE 46: Builder: Constructing Complex Objects Step by Step

SLIDE 47: The Telescoping Constructor Anti-Pattern

SLIDE 47B: Worked Example: Building a User Profile with Validation

SLIDE 48: When Builder Is Overkill vs When It Is Essential

SLIDE 48B: Interview Practice: Builder

SLIDE 49: Singleton: When Genuinely Appropriate and Why It Is Overused

SLIDE 49B: Singleton: Thread Safety and Testing Difficulties

SLIDE 50: Registry Alternative to Singleton

SLIDE 50B: Note It Down: Creational Pattern Signals

SLIDE 51: Creational Pattern Decision Framework

SLIDE 52: Creational Patterns Interview Practice and Common Mistakes

SLIDE 53: Section Summary and Cheat Sheet

SLIDE 41: Section Intro: Creational Design Patterns

Section 4: Creational Design Patterns

Creational patterns address a specific problem: how and where objects get created. In a small codebase, calling `new SomeClass()` everywhere is fine. In a larger system, scattered object creation leads to tight coupling, duplicated configuration logic, and classes that are impossible to test in isolation. Creational patterns centralize and control object creation so that the rest of your code depends on abstractions rather than concrete classes. This is the Dependency Inversion Principle from Section 3 applied specifically to construction.

What We Will Cover

1. Factory Method: decoupling object creation from usage
2. Abstract Factory: creating families of related objects consistently
3. Builder: constructing complex objects step by step
4. Singleton: when it is genuinely appropriate and why it is overused
5. Creational pattern decision framework

Where Creational Patterns Appear in LLD Interviews

Interview Signal	Creational Pattern	Why
“Support multiple types of X”	Factory Method	Decouple creation from the calling code
“These objects must be created together consistently”	Abstract Factory	Ensure compatible object families
“This object has 8+ constructor parameters”	Builder	Avoid telescoping constructors
“We need exactly one instance of X”	Singleton (cautiously)	Controlled shared state
“How would you add a new type without changing existing code?”	Factory Method	OCP applied to object creation

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

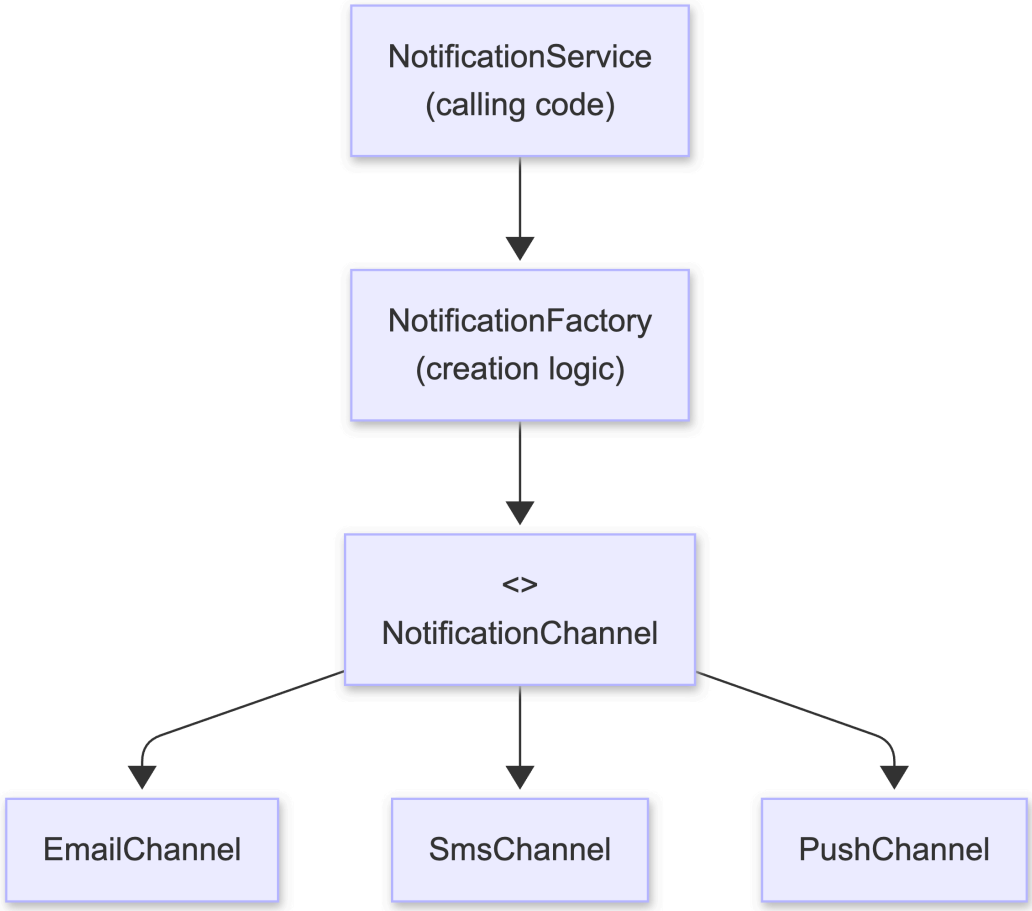
SLIDE 42: Factory Method: Decoupling Object Creation from Usage

The Factory Method pattern moves object creation out of the calling code and into a dedicated method or class. The calling code requests an object through the factory and receives it through an interface, without knowing which concrete class was instantiated.

Why This Matters

When creation logic is scattered across the codebase, every class that creates objects is coupled to the concrete implementations. If you have 15 classes that each call `new EmailNotification()`, switching from email to push notifications requires editing 15 files. A factory centralizes this decision into one place.

The Core Idea



The `NotificationService` never calls `new EmailChannel()`. It asks the `NotificationFactory` for a channel, and the factory decides which concrete class to instantiate based on configuration, user preferences, or business rules. The service works with the `NotificationChannel` interface and is completely decoupled from the concrete implementations.

Interview Relevance: Factory Method is the most commonly tested creational pattern. Interviewers use it to evaluate whether you understand OCP applied to object creation: adding a new type means updating the factory, not every class that uses the type.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 43: Wrong Approach: Switch Statement in Calling Code

Here is what happens when creation logic lives directly in the calling code.

```
public class NotificationService {
    public void sendNotification(String userId, String messag
e, String channelType) {
        if (channelType.equals("email")) {
            EmailClient emailClient = new EmailClient("smtp.p
rovider.com", 587);
            String address = UserDirectory.lookupEmail(userI
d);
            emailClient.send(address, "Notification", messag
e);
        } else if (channelType.equals("sms")) {
            SmsGateway smsGateway = new SmsGateway("api-key-1
23");
            String phoneNumber = UserDirectory.lookupPhone(us
erId);
            smsGateway.sendText(phoneNumber, message);
        } else if (channelType.equals("push")) {
            PushService pushService = new PushService("fireba
se-key");
            String deviceToken = DeviceRegistry.lookupToken(u
serId);
            pushService.sendPush(deviceToken, message);
        }
    }
}
```

What Is Wrong

Problem	Impact
Adding WhatsApp requires editing this class	OCP violation
`NotificationService` knows SMTP ports, API keys, Firebase config	SRP violation: business logic mixed with infrastructure details

Problem	Impact
Cannot test notification logic without real email/SMS/push services	DIP violation: depends on concrete implementations
Every class that sends notifications duplicates this switch logic	Shotgun surgery: one change affects many files
Channel-specific setup (API keys, addresses) is hardcoded	Configuration is scattered, not centralized

This code violates three SOLID principles simultaneously. The factory method fixes all of them in one refactoring step.

Interview Relevance: Interviewers often present code like this and ask "what would you improve?" Identifying the multiple SOLID violations and proposing a factory is a strong response.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 43B: Right Approach: Factory Method Implementation

The Interface

```
public interface NotificationChannel {
    void send(String userId, String message);
}
```

Concrete Implementations

```
public class EmailChannel implements NotificationChannel {
    private final EmailClient emailClient;

    public EmailChannel(EmailClient emailClient) {
        this.emailClient = emailClient;
    }

    @Override
```

```
    public void send(String userId, String message) {
        String address = UserDirectory.lookupEmail(userId);
        emailClient.send(address, "Notification", message);
    }
}

public class SmsChannel implements NotificationChannel {
    private final SmsGateway smsGateway;

    public SmsChannel(SmsGateway smsGateway) {
        this.smsGateway = smsGateway;
    }

    @Override
    public void send(String userId, String message) {
        String phoneNumber = UserDirectory.lookupPhone(userI
d);
        smsGateway.sendText(phoneNumber, message);
    }
}
```

The Factory

```
public class NotificationChannelFactory {
    private final Map<String, NotificationChannel> channels;

    public NotificationChannelFactory(Map<String, Notificatio
nChannel> channels) {
        this.channels = channels;
    }

    public NotificationChannel create(String channelType) {
        NotificationChannel channel = channels.get(channelTyp
e);
        if (channel == null) {
            throw new UnsupportedChannelException("Unknown ch
```

```
annel: " + channelType);
        }
        return channel;
    }
}
```

The Calling Code

```
public class NotificationService {
    private final NotificationChannelFactory channelFactory;

    public NotificationService(NotificationChannelFactory cha
nnelFactory) {
        this.channelFactory = channelFactory;
    }

    public void sendNotification(String userId, String messag
e, String channelType) {
        NotificationChannel channel = channelFactory.create(c
hannelType);
        channel.send(userId, message);
    }
}
```

`NotificationService` is now three lines of business logic. It knows nothing about email servers, SMS gateways, or push services. Adding WhatsApp means writing a `WhatsAppChannel` class and registering it in the factory's map. Zero changes to `NotificationService`.

Interview Tip: In an LLD interview, the factory does not need to be a separate class. A static factory method on the interface itself works for simple cases. The key decision the interviewer evaluates is whether creation logic is centralized and the calling code is decoupled from concrete types.

Interview Relevance: Show the before (switch statement) and after (factory) in your interview to demonstrate that you understand why the pattern exists, not just how to implement it.

SLIDE 44: Worked Example: Notification Channel Creation

Here is how the factory-based notification system is wired together at application startup for a platform like Slack.

Application Configuration

```
public class NotificationModule {
    public static NotificationService create(AppConfig confi
g) {
        Map<String, NotificationChannel> channels = new HashM
ap<>();

        channels.put("email", new EmailChannel(
            new EmailClient(config.getSmtpHost(), config.getS
mtpPort())
        ));
        channels.put("sms", new SmsChannel(
            new SmsGateway(config.getSmsApiKey())
        ));
        channels.put("push", new PushChannel(
            new PushService(config.getFirebaseKey())
        ));

        NotificationChannelFactory factory = new Notification
ChannelFactory(channels);
        return new NotificationService(factory);
    }
}
```

Adding a New Channel (WhatsApp)

Step 1: Write the new implementation.

```
public class WhatsAppChannel implements NotificationChannel {
    private final WhatsAppClient whatsAppClient;

    public WhatsAppChannel(WhatsAppClient whatsAppClient) {
        this.whatsAppClient = whatsAppClient;
    }

    @Override
    public void send(String userId, String message) {
        String phoneNumber = UserDirectory.lookupPhone(userI
d);
        whatsAppClient.sendMessage(phoneNumber, message);
    }
}
```

Step 2: Register it in the configuration.

```
channels.put("whatsapp", new WhatsAppChannel(
    new WhatsAppClient(config.getWhatsAppApiKey())
));
```

Two changes: one new class, one new line in the configuration module. `NotificationService`, `NotificationChannelFactory`, and all existing channel implementations are untouched.

Interview Relevance: Walking through the “add a new type” scenario step by step in an interview is one of the strongest ways to demonstrate OCP compliance. Count the files changed (one new file, one line in configuration) and state it explicitly.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 45: Abstract Factory: Families of Related Objects

The Abstract Factory pattern creates families of related objects that must be used together. While Factory Method creates one type of object, Abstract Factory creates a set of objects that are designed to work as a group.

When You Need Abstract Factory

Consider a UI component library that supports multiple themes. Each theme defines its own versions of buttons, text fields, and dropdowns. A “Material” theme produces Material-styled components. A “Fluent” theme produces Microsoft Fluent-styled components. Mixing a Material button with a Fluent dropdown looks broken. The components within a theme must be created together consistently.

The Structure

```
public interface UIComponentFactory {
    Button createButton(String label);
    TextField createTextField(String placeholder);
    Dropdown createDropdown(List<String> options);
}

public class MaterialUIFactory implements UIComponentFactory
{
    @Override
    public Button createButton(String label) {
        return new MaterialButton(label);
    }

    @Override
    public TextField createTextField(String placeholder) {
        return new MaterialTextField(placeholder);
    }

    @Override
    public Dropdown createDropdown(List<String> options) {
        return new MaterialDropdown(options);
    }
}
```

```
public class FluentUIFactory implements UIComponentFactory {
    @Override
    public Button createButton(String label) {
        return new FluentButton(label);
    }

    @Override
    public TextField createTextField(String placeholder) {
        return new FluentTextField(placeholder);
    }

    @Override
    public Dropdown createDropdown(List<String> options) {
        return new FluentDropdown(options);
    }
}
```

The Calling Code

```
public class FormRenderer {
    private final UIComponentFactory componentFactory;

    public FormRenderer(UIComponentFactory componentFactory)
{
        this.componentFactory = componentFactory;
    }

    public Form renderLoginForm() {
        Button submitButton = componentFactory.createButton
("Sign In");
        TextField emailField = componentFactory.createTextFie
ld("Enter email");
        TextField passwordField = componentFactory.createText
Field("Enter password");
        return new Form(emailField, passwordField, submitButt
```

```
on);
    }
}
```

`FormRenderer` does not know which theme is active. It creates components through the factory, and the factory guarantees that all components belong to the same theme. Switching from Material to Fluent means injecting a different factory. Zero changes to `FormRenderer`.

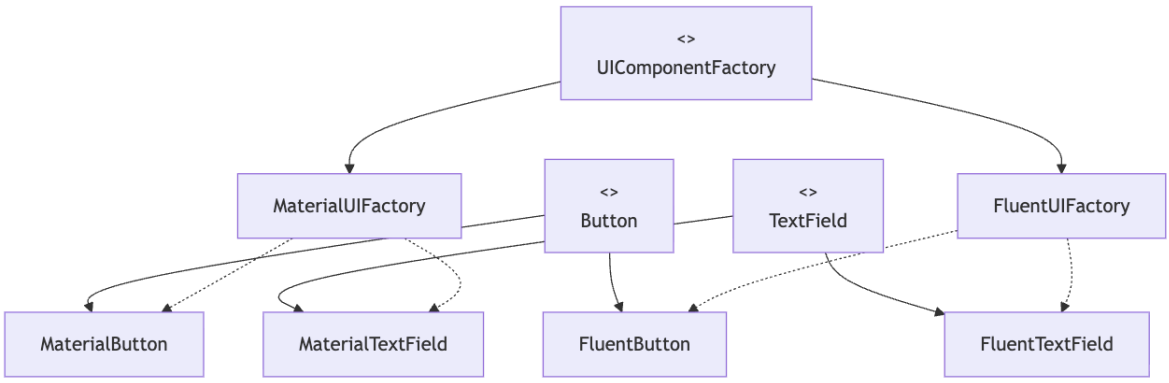
Interview Relevance: Abstract Factory is less commonly tested than Factory Method, but it appears in any LLD question involving themes, platform-specific rendering, or environment-specific configurations (dev vs staging vs production).

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 45B: Worked Example: UI Component Factory for Themes

The Complete Object Hierarchy



Each factory creates components from its own family (solid lines = inheritance, dashed lines = creates). The calling code depends only on the interfaces (`UIComponentFactory`, `Button`, `TextField`), never on the concrete classes.

Factory Method vs Abstract Factory

Dimension	Factory Method	Abstract Factory
Creates	One type of object	A family of related objects
Interface	One creation method	Multiple creation methods
Solves	“Which concrete class to instantiate”	“Which family of classes to use together”
Example	`createNotificationChannel("email")`	`createButton()`, `createTextField()`, `createDropdown()` as a set
Guarantee	Correct single object	All objects are from the same family
Complexity	Lower	Higher

When to Use Abstract Factory

Use Abstract Factory when objects must be created as a consistent group and mixing objects from different groups would be incorrect. If each object can be created independently without consistency concerns, use individual Factory Methods instead.

Interview Tip: In an interview, do not reach for Abstract Factory unless you see the “family of related objects” signal. Using Abstract Factory for a single object type is over-engineering, and interviewers will flag it. Ask yourself: “Do these objects need to be consistent with each other?” If no, Factory Method is sufficient.

Interview Relevance: The key interview question is “why Abstract Factory instead of Factory Method?” The answer is always about consistency across a family of related objects.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 45C: Interview Practice: Factory Method and Abstract Factory

Scenario

You are designing a document generation system for a platform like DocuSign. The system creates documents in multiple formats (PDF, Word, HTML). Each

format requires a header renderer, a body renderer, and a footer renderer that are specific to that format. The interviewer asks: “How would you structure the creation of these renderers?”

Pause the video and decide which creational pattern to use for 60 seconds before continuing.

Weak Answer

“I would use a Factory Method that takes a format string and returns a renderer.”

This answer misses the core problem. There are three renderers (header, body, footer) per format, and they must be consistent. A single Factory Method returns one object, not a coordinated set. Using three independent Factory Methods risks mixing a PDF header renderer with an HTML body renderer.

Strong Answer

“Since each format requires a consistent family of renderers (header, body, footer), I would use Abstract Factory. A `DocumentRendererFactory` interface defines `createHeaderRenderer()`, `createBodyRenderer()`, and `createFooterRenderer()`. `PdfRendererFactory` creates all three PDF variants. `HtmlRendererFactory` creates all three HTML variants.

The document generation service depends on `DocumentRendererFactory`. Switching formats means injecting a different factory. The service never mixes renderers from different formats because the factory guarantees consistency.”

The strong answer identifies the “family of related objects” signal, names the correct pattern, and explains why Factory Method alone is insufficient.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 46: Builder: Constructing Complex Objects Step by Step

The Builder pattern separates the construction of a complex object from its representation, allowing the same construction process to create different configurations. Builder is the right pattern when an object has many parameters, most of which are optional, and the valid combinations are too numerous to handle with constructor overloading.

When Builder Is Appropriate

Builder earns its place when an object has more than 4-5 constructor parameters, many of which are optional, and the order of parameter specification does not matter. The classic example is any configuration object, query object, or request object with a mix of required and optional fields.

The Pattern

```
public class SearchQuery {
    private final String searchTerm;
    private final int maxResults;
    private final SortOrder sortOrder;
    private final List<String> filters;
    private final LocalDate fromDate;
    private final LocalDate toDate;
    private final boolean includeArchived;

    private SearchQuery(Builder builder) {
        this.searchTerm = builder.searchTerm;
        this.maxResults = builder.maxResults;
        this.sortOrder = builder.sortOrder;
        this.filters = List.copyOf(builder.filters);
        this.fromDate = builder.fromDate;
        this.toDate = builder.toDate;
        this.includeArchived = builder.includeArchived;
    }

    public static class Builder {
        private final String searchTerm;
        private int maxResults = 20;
        private SortOrder sortOrder = SortOrder.RELEVANCE;
        private List<String> filters = new ArrayList<>();
        private LocalDate fromDate = null;
        private LocalDate toDate = null;
        private boolean includeArchived = false;
```

```
        public Builder(String searchTerm) {
            this.searchTerm = searchTerm;
        }

        public Builder maxResults(int maxResults) {
            this.maxResults = maxResults;
            return this;
        }

        public Builder sortOrder(SortOrder sortOrder) {
            this.sortOrder = sortOrder;
            return this;
        }

        public Builder addFilter(String filter) {
            this.filters.add(filter);
            return this;
        }

        public Builder dateRange(LocalDate from, LocalDate t
o) {
            this.fromDate = from;
            this.toDate = to;
            return this;
        }

        public Builder includeArchived(boolean include) {
            this.includeArchived = include;
            return this;
        }

        public SearchQuery build() {
            return new SearchQuery(this);
        }
```

```
    }
}
```

Usage

```
SearchQuery query = new SearchQuery.Builder("system design")
    .maxResults(50)
    .sortOrder(SortOrder.DATE)
    .addFilter("category:engineering")
    .dateRange(LocalDate.of(2025, 1, 1), LocalDate.now())
    .includeArchived(true)
    .build();
```

Each method call is self-documenting. The reader knows exactly what each parameter means without checking parameter positions. Optional parameters are simply omitted.

Interview Relevance: Builder appears in LLD interviews when you design objects with many configuration options. Interviewers look for whether you recognize the “too many constructor parameters” signal and reach for Builder instead of telescoping constructors.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 47: The Telescoping Constructor Anti-Pattern

The telescoping constructor is the problem that Builder solves. It occurs when a class needs to support many combinations of optional parameters, and the developer creates multiple constructor overloads to handle them.

```
public class SearchQuery {
    public SearchQuery(String searchTerm) {
        this(searchTerm, 20);
    }
```

```
    public SearchQuery(String searchTerm, int maxResults) {
        this(searchTerm, maxResults, SortOrder.RELEVANCE);
    }

    public SearchQuery(String searchTerm, int maxResults, Sor
tOrder sortOrder) {
        this(searchTerm, maxResults, sortOrder, Collections.e
mptyList());
    }

    public SearchQuery(String searchTerm, int maxResults, Sor
tOrder sortOrder,
                       List<String> filters) {
        this(searchTerm, maxResults, sortOrder, filters, nul
l, null);
    }

    public SearchQuery(String searchTerm, int maxResults, Sor
tOrder sortOrder,
                       List<String> filters, LocalDate fromDa
te, LocalDate toDate) {
        this(searchTerm, maxResults, sortOrder, filters, from
Date, toDate, false);
    }

    public SearchQuery(String searchTerm, int maxResults, Sor
tOrder sortOrder,
                       List<String> filters, LocalDate fromDa
te, LocalDate toDate,
                       boolean includeArchived) {
        this.searchTerm = searchTerm;
        this.maxResults = maxResults;
        this.sortOrder = sortOrder;
        this.filters = filters;
        this.fromDate = fromDate;
        this.toDate = toDate;
```

```
        this.includeArchived = includeArchived;
    }
}
```

Why This Is Harmful

Problem	Impact
Parameter position confusion	`new SearchQuery("design", 50, SortOrder.DATE, filters, null, null, true)` requires remembering that `true` is the 7th parameter and means "include archived"
Cannot skip optional parameters	To set `includeArchived` without `fromDate`, you must pass `null, null` as placeholders
Every new optional parameter adds a new constructor	Growth is linear with parameter count, and existing callers may need updating
Code is unreadable at the call site	`new SearchQuery("x", 20, null, null, null, null, true)` gives no indication what each `null` means

With 7 parameters, there are 127 possible combinations of optional values. Telescoping constructors cannot handle this without an explosion of overloads.

Interview Relevance: Recognizing the telescoping constructor anti-pattern and proposing Builder as the fix demonstrates that you understand when patterns solve real problems rather than adding unnecessary abstraction.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 47B: Worked Example: Building a User Profile with Validation

Here is a Builder that enforces validation rules during construction, which demonstrates how Builder can protect invariants (connecting back to encapsulation from Section 2).

```
public class UserProfile {
    private final String userId;
```

```
    private final String displayName;
    private final String email;
    private final String bio;
    private final String avatarUrl;
    private final Locale preferredLanguage;
    private final ZoneId timezone;
    private final boolean emailNotificationsEnabled;
    private final boolean profilePublic;

    private UserProfile(Builder builder) {
        this.userId = builder.userId;
        this.displayName = builder.displayName;
        this.email = builder.email;
        this.bio = builder.bio;
        this.avatarUrl = builder.avatarUrl;
        this.preferredLanguage = builder.preferredLanguage;
        this.timezone = builder.timezone;
        this.emailNotificationsEnabled = builder.emailNotific
ationsEnabled;
        this.profilePublic = builder.profilePublic;
    }

    public static class Builder {
        private final String userId;
        private final String displayName;
        private final String email;
        private String bio = "";
        private String avatarUrl = null;
        private Locale preferredLanguage = Locale.ENGLISH;
        private ZoneId timezone = ZoneId.of("UTC");
        private boolean emailNotificationsEnabled = true;
        private boolean profilePublic = true;

        public Builder(String userId, String displayName, Str
ing email) {
            if (userId == null || userId.isBlank()) {
```

```
                throw new IllegalArgumentException("User ID i
s required");
            }
            if (email == null || !email.contains("@")) {
                throw new IllegalArgumentException("Valid ema
il is required");
            }
            this.userId = userId;
            this.displayName = displayName;
            this.email = email;
        }

        public Builder bio(String bio) {
            if (bio.length() > 500) {
                throw new IllegalArgumentException("Bio canno
t exceed 500 characters");
            }
            this.bio = bio;
            return this;
        }

        public Builder timezone(ZoneId timezone) {
            this.timezone = timezone;
            return this;
        }

        public Builder emailNotifications(boolean enabled) {
            this.emailNotificationsEnabled = enabled;
            return this;
        }

        public Builder profileVisibility(boolean isPublic) {
            this.profilePublic = isPublic;
            return this;
        }
```

```
        public UserProfile build() {
            return new UserProfile(this);
        }
    }
}
```

Usage

```
UserProfile profile = new UserProfile.Builder("user_42", "Mukul", "mukul@example.com")
    .bio("Software engineer interested in distributed systems")
    .timezone(ZoneId.of("America/Toronto"))
    .emailNotifications(false)
    .build();
```

Required parameters (`userId`, `displayName`, `email`) are enforced in the Builder constructor. Optional parameters have sensible defaults. Validation happens during construction, so an invalid `UserProfile` can never exist.

Interview Tip: When you use Builder in an LLD interview, put required parameters in the Builder constructor and optional parameters as chained methods with defaults. This shows the interviewer that you understand the distinction between mandatory and optional fields and that you use the type system to enforce it.

Interview Relevance: Builder with validation demonstrates encapsulation (Section 2) applied to construction. The object cannot be created in an invalid state, which is exactly what interviewers evaluate.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 48: When Builder Is Overkill vs When It Is Essential

Builder adds a significant amount of boilerplate code: an inner class, chained setter methods, a build method, and a private constructor on the outer class. This cost is justified only when the object is genuinely complex.

Builder Is Essential When

1. The object has 5+ parameters with many optional combinations
2. The object has validation rules that span multiple fields (e.g., `endDate` must be after `startDate`)
3. The object should be immutable once constructed (all fields `final`, no setters)
4. The same construction process needs to create different representations

Builder Is Overkill When

1. The object has 2-3 parameters, all required
2. There are no optional fields or variant configurations
3. A simple constructor is clear and readable

Decision Framework

Parameters	Optional Fields	Recommended Approach
1-3	None	Simple constructor
3-4	1-2	Constructor overloading (if clear) or Builder
5+	2+	Builder
Any count	Cross-field validation	Builder (validation in `build()` method)
Any count	Object must be immutable	Builder (private constructor, final fields)

Common Mistake

Using Builder for a class with two required fields and no optional fields. This adds 30+ lines of boilerplate for zero benefit. The interviewer will flag this as over-engineering.

Interview Relevance: Knowing when not to use a pattern is as important as knowing how to use it. If an interviewer asks "would you use Builder here?" for a simple 2-field class, the correct answer is "no, a simple constructor is sufficient."

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 48B: Interview Practice: Builder

Scenario

You are designing an `HttpRequest` class for an HTTP client library. An HTTP request has a URL (required), an HTTP method (required), headers (optional, zero or more), a request body (optional), a timeout in milliseconds (optional, default 30000), and a retry count (optional, default 3). The interviewer asks: “How would you design the construction of this object?”

Pause the video and design the construction approach for 60 seconds before continuing.

Weak Answer

```
public class HttpRequest {
    private final String url;
    private final String method;
    private final Map<String, String> headers;
    private final String body;
    private final int timeoutMs;
    private final int retryCount;

    public HttpRequest(String url, String method, Map<String,
String> headers,
                       String body, int timeoutMs, int retryC
ount) {
        this.url = url;
        this.method = method;
        this.headers = headers;
        this.body = body;
        this.timeoutMs = timeoutMs;
        this.retryCount = retryCount;
    }
}
```

Six parameters with four optional ones. Callers must pass `null` for unused parameters and remember the exact parameter order. `new`

`HttpRequest("https://api.example.com", "GET", null, null, 30000, 3)` is unreadable.

Strong Answer

```
HttpRequest request = new HttpRequest.Builder("https://api.ex
ample.com", HttpMethod.GET)
    .addHeader("Authorization", "Bearer token123")
    .addHeader("Content-Type", "application/json")
    .body("{\"key\":\"value\"}")
    .timeoutMs(5000)
    .build();
```

Required parameters (`url`, `method`) in the Builder constructor. Optional parameters chained with clear method names. Retry count defaults to 3 without needing to be specified. The `build()` method can validate that a `POST` request has a body while a `GET` request does not.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 49: Singleton: When Genuinely Appropriate and Why It Is Overused

The Singleton pattern ensures that a class has exactly one instance and provides a global point of access to it. It is the most controversial design pattern because it is simultaneously simple to implement and easy to misuse.

When Singleton Is Genuinely Appropriate

Singleton is justified when creating multiple instances would cause correctness or resource problems:

1. **Connection pools:** Multiple pools to the same database waste connections and bypass pool size limits
2. **Configuration registries:** Application configuration loaded once from a file or environment should not be re-parsed by every class that needs a setting

3. **Hardware interface controllers:** A printer spooler or device driver that maps to a single physical resource

In each case, the “exactly one instance” constraint is a correctness requirement, not a convenience choice.

Why Singleton Is Overused

Developers reach for Singleton when they want a class to be easily accessible from anywhere in the codebase. This is using Singleton as a replacement for dependency injection, which causes three problems:

Problem	Why It Matters
Hidden dependencies	Classes that call `DatabasePool.getInstance()` have an invisible dependency that does not appear in their constructor
Tight coupling	Every class that uses the singleton is coupled to the concrete implementation, violating DIP
Testing difficulty	Cannot substitute a mock in unit tests because the singleton controls its own lifecycle
Shared mutable state	Global state accessed from multiple threads creates concurrency bugs

The Rule of Thumb

If the reason for using Singleton is “I need easy access to this object,” the real solution is dependency injection, not Singleton. If the reason is “creating two instances would cause a bug,” Singleton may be appropriate.

Interview Relevance: Interviewers often ask about Singleton to test whether you understand its trade-offs. Saying “I would use Singleton for the database connection pool” is acceptable. Saying “I would use Singleton for the UserService” signals a misunderstanding of the pattern.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 49B: Singleton: Thread Safety and Testing Difficulties

The Naive Implementation (Not Thread-Safe)

```
public class ConnectionPool {
    private static ConnectionPool instance;

    private final List<Connection> pool = new ArrayList<>();

    private ConnectionPool() {
        for (int i = 0; i < 10; i++) {
            pool.add(DataSource.createConnection());
        }
    }

    public static ConnectionPool getInstance() {
        if (instance == null) {
            instance = new ConnectionPool();
        }
        return instance;
    }
}
```

If two threads call `getInstance()` simultaneously when `instance` is null, both threads pass the null check, and two instances are created. The second assignment overwrites the first, and any thread holding a reference to the first instance now has a stale, potentially broken object.

Thread-Safe Approaches

Approach	Code	Trade-Off
Synchronized method	`public static synchronized ConnectionPool getInstance()`	Simple but every call pays the synchronization cost, even after initialization
Double-checked locking	Check null, synchronize, check null again, create	Correct with `volatile` keyword, but complex and error-prone
Eager initialization	`private static final ConnectionPool INSTANCE = new ConnectionPool();`	Simple and thread-safe, but instance is created even if never used

Approach	Code	Trade-Off
Enum singleton	`public enum ConnectionPool { INSTANCE; }`	Recommended by Joshua Bloch, handles serialization and reflection attacks

The Enum Approach (Recommended in Java)

```
public enum ConnectionPool {
    INSTANCE;

    private final List<Connection> connections = new ArrayLis
t<>();

    ConnectionPool() {
        for (int i = 0; i < 10; i++) {
            connections.add(createConnection());
        }
    }

    public synchronized Connection acquire() {
        if (connections.isEmpty()) {
            throw new PoolExhaustedException("No available co
nnections");
        }
        return connections.remove(connections.size() - 1);
    }

    public synchronized void release(Connection connection) {
        connections.add(connection);
    }
}
```

The JVM guarantees that enum values are instantiated exactly once, in a thread-safe manner, and are immune to reflection-based attacks that can bypass private constructors.

The Testing Problem

Regardless of the thread safety approach, Singleton makes unit testing difficult. A test that uses `ConnectionPool.getInstance()` cannot substitute a mock. The test either needs a real connection pool or must use reflection to reset the static field between tests, which is fragile and error-prone. This is the fundamental tension with Singleton: it trades testability for global access.

Interview Relevance: If you use Singleton in your LLD answer, mention thread safety and the testing trade-off proactively. This shows the interviewer that you understand the pattern's costs, not just its mechanics.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 50: Registry Alternative to Singleton

When you need a single instance of a class but want to avoid the testing and coupling problems of Singleton, use a registry or dependency injection container. The instance is created once during application startup and passed to every class that needs it through constructor injection.

Singleton Approach (Tight Coupling)

```
public class OrderService {
    public void placeOrder(Order order) {
        ConnectionPool pool = ConnectionPool.getInstance();
        Connection conn = pool.acquire();
        PreparedStatement stmt = conn.prepareStatement("INSER
T INTO orders VALUES (?, ?)");
        stmt.setString(1, order.getId());
        stmt.setDouble(2, order.getTotal());
        stmt.executeUpdate();
        pool.release(conn);
    }
}
```

`OrderService` has a hidden dependency on `ConnectionPool`. It controls when it accesses the pool, and there is no way to substitute a different pool for testing.

Registry/DI Approach (Loose Coupling)

```
public class OrderService {
    private final ConnectionPool connectionPool;

    public OrderService(ConnectionPool connectionPool) {
        this.connectionPool = connectionPool;
    }

    public void placeOrder(Order order) {
        Connection conn = connectionPool.acquire();
        PreparedStatement stmt = conn.prepareStatement("INSER
T INTO orders VALUES (?, ?)");
        stmt.setString(1, order.getId());
        stmt.setDouble(2, order.getTotal());
        stmt.executeUpdate();
        connectionPool.release(conn);
    }
}
```

Application Startup (the Composition Root)

```
public class Application {
    public static void main(String[] args) {
        ConnectionPool connectionPool = new ConnectionPool(1
0);
        OrderRepository orderRepository = new PostgresOrderRe
pository(connectionPool);
        PaymentProcessor paymentProcessor = new StripePayment
Processor(stripeConfig);
        OrderService orderService = new OrderService(connecti
onPool);

        WebServer server = new WebServer(orderService);
        server.start(8080);
```

```
    }
}
```

The `ConnectionPool` is created once (effectively a singleton in behavior) but passed explicitly through constructors. Every class declares its dependencies visibly. Tests can inject a `FakeConnectionPool`. No global state, no hidden coupling, no thread safety concerns beyond the pool itself.

When to Still Use Singleton

If the class is genuinely infrastructure-level (logging framework, metrics collector) and injecting it through 50+ constructors would add noise without benefit, Singleton remains pragmatic. But for business logic dependencies, the registry/DI approach is almost always better.

Interview Tip: In an LLD interview, prefer dependency injection over Singleton for any dependency in your class diagram. If the interviewer asks "should this be a Singleton?", answer with: "I would create one instance at startup and inject it. This gives us the same single-instance behavior with the added benefit of testability."

Interview Relevance: Choosing DI over Singleton in your class diagram signals senior-level thinking. Interviewers view unnecessary Singletons as a sign that the candidate does not understand dependency management.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 50B: Note It Down: Creational Pattern Signals

These are the signals that tell you which creational pattern to reach for during an LLD interview. Memorize the mapping from signal to pattern.

Signal-to-Pattern Quick Reference

Signal in the Requirements	Pattern	Why
"Create objects of different types based on configuration or input"	Factory Method	Decouple creation decision from calling code

Signal in the Requirements	Pattern	Why
“Create a set of objects that must be consistent with each other”	Abstract Factory	Guarantee family consistency
“Object has 5+ parameters with many optional”	Builder	Avoid telescoping constructors, self-documenting construction
“Object needs cross-field validation during construction”	Builder	Validate in `build()`, reject invalid states
“Exactly one instance for correctness reasons”	Singleton (cautiously)	Connection pools, hardware controllers
“I need global access to a shared service”	Not Singleton, use Dependency Injection	Testability, visible dependencies, loose coupling

Three Questions to Ask Yourself

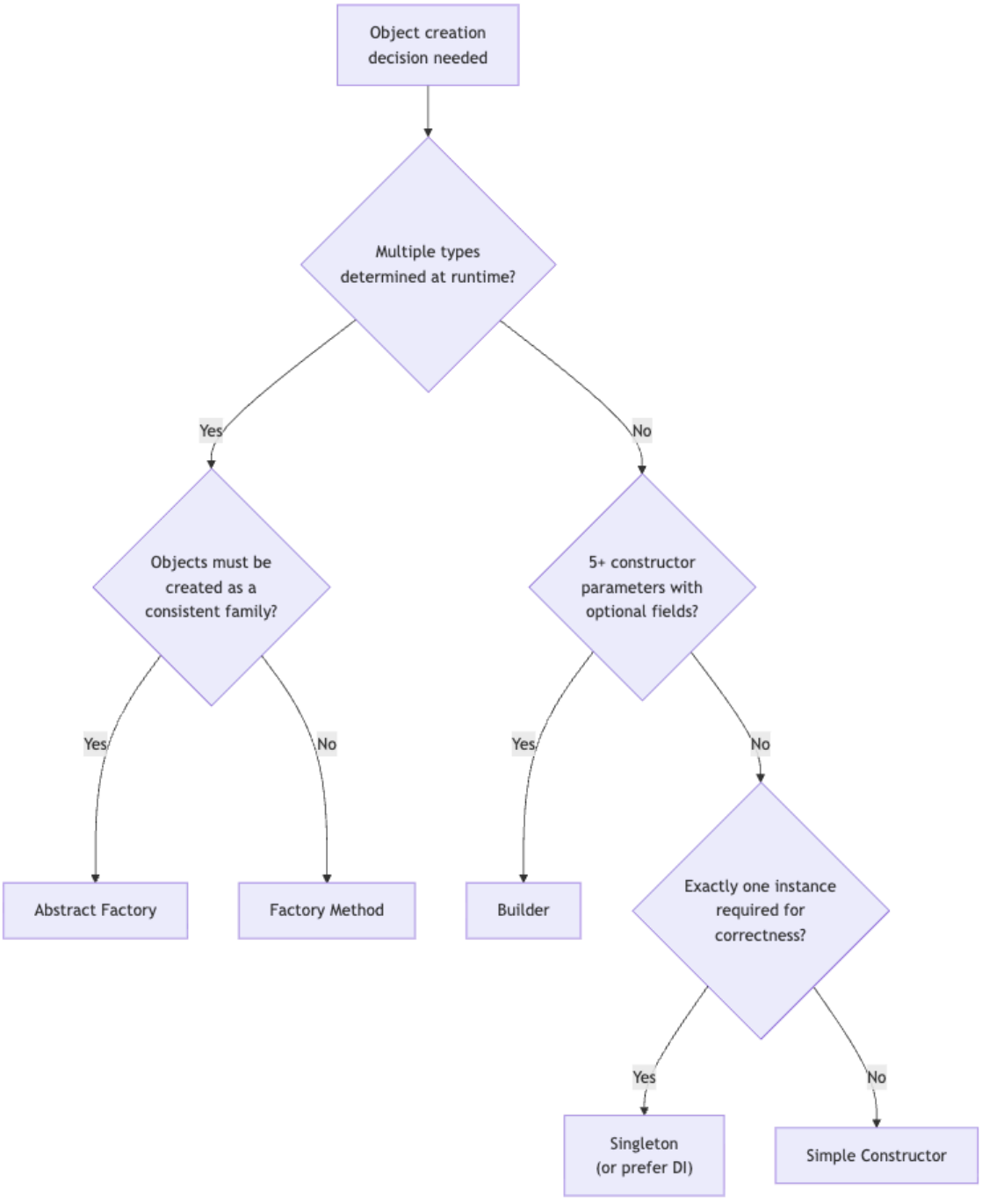
1. **How many types?** If the answer is “multiple types determined at runtime,” use Factory Method.
2. **How many parameters?** If the answer is “5+ with optional combinations,” use Builder.
3. **How many instances?** If the answer is “exactly one for correctness,” consider Singleton but prefer DI.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 51: Creational Pattern Decision Framework

Use this decision tree when choosing a creational pattern in an LLD interview.



Common Combinations

Creational patterns are often used together. A Factory Method might use a Builder internally to construct complex objects. An Abstract Factory might return Singletons for expensive-to-create components.

Combination	When It Appears
Factory Method + Builder	Factory decides the type, Builder handles complex construction
Abstract Factory + Singleton	Factory itself is a Singleton (one factory per theme/platform)
Factory Method + Strategy (Section 6)	Factory creates Strategy objects based on configuration
Builder + Validation	Builder's `build()` method enforces cross-field invariants

Interview Relevance: Having this decision tree internalized means you can justify your creational pattern choice in under 10 seconds during an interview. "The object has 8 optional parameters, so I am using Builder" is a concise, correct justification.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 52: Creational Patterns Interview Practice and Common Mistakes

Practice Scenario

You are designing a logging framework for a platform like Datadog. The framework supports multiple output destinations (console, file, remote server). Each destination has different configuration options: console needs a color scheme, file needs a path and rotation policy, remote needs a URL and authentication token. The logger itself should exist as a single instance per application. The interviewer asks: "Walk me through your class design for creating these loggers."

Pause the video and design the creation structure for 90 seconds before continuing.

Key Design Decisions

The output destinations are different types created based on configuration, which is the Factory Method signal. Each destination has different construction parameters, and the file destination has 4+ optional fields (path, rotation size, rotation count, compression), which is the Builder signal for that specific class. The logger itself being a single instance is the Singleton signal, but dependency injection is the stronger approach.

A strong answer: Factory Method to create destination instances, Builder for the complex `FileDestination` configuration, and DI (not Singleton) for the `Logger` instance.

Common Creational Pattern Mistakes

Mistake	Why It Is Wrong	Stronger Approach
Using Singleton for every shared service	Creates hidden dependencies, blocks testability	Dependency injection through constructors
Using Builder for a 2-field class	Over-engineering adds 30+ lines of boilerplate for no benefit	Simple constructor
Using Factory Method when there is only one type	Unnecessary indirection with no extensibility benefit	Direct constructor call
Putting creation logic in the calling class	Violates SRP, couples caller to all concrete types	Extract into a Factory
Using Abstract Factory when objects are independent	Adds complexity without consistency benefit	Individual Factory Methods
Making every class a Singleton “just in case”	Global mutable state, testing nightmares, hidden coupling	Create once, inject everywhere

Interview Relevance: These mistakes map directly to the “over-engineering” and “under-engineering” signals interviewers evaluate. Patterns should solve specific problems, not be applied as default practice.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 53: Section Summary and Cheat Sheet

Concepts Covered

Pattern	One-Line Summary
Factory Method	Centralizes object creation behind a factory, decoupling callers from concrete types
Abstract Factory	Creates families of related objects that must be used together consistently
Builder	Constructs complex objects step by step with optional parameters and validation
Singleton	Ensures exactly one instance, but prefer DI for most use cases
Telescoping Constructor	Anti-pattern where constructor overloads grow unmanageably (solved by Builder)
Registry/DI	Alternative to Singleton that provides testability and visible dependencies

Common Interview Mistakes

Mistake	Stronger Approach
“I used Singleton because the service is shared”	“I create one instance at startup and inject it through constructors for testability”
“I used Builder for the User class” (which has 2 required fields)	“A simple constructor is sufficient. Builder is for objects with many optional parameters”
“I put the creation switch in OrderService”	“I extracted a Factory so OrderService depends on the interface, not the creation logic”
“I used Abstract Factory for creating notifications”	“Since notification channels are independent (not a consistent family), Factory Method is sufficient”

Key Points to Remember

Pattern	Recognition Signal	Justification Phrase
Factory Method	“Multiple types determined at runtime”	“Adding a new type means writing one new class and updating the factory”
Abstract Factory	“Objects must be consistent as a family”	“The factory guarantees all components belong to the same theme/platform”
Builder	“5+ parameters with optional combinations”	“Builder provides self-documenting construction and validates invariants”

Pattern	Recognition Signal	Justification Phrase
Singleton	“Exactly one for correctness”	“I prefer DI for the same single-instance behavior with better testability”

How This Connects to Other Sections

- **Section 2 (OOP):** Factory Method relies on polymorphism. Builder protects encapsulation by enforcing invariants during construction.
- **Section 3 (SOLID):** Factory Method implements OCP for object creation. Builder applies SRP by separating construction from the object itself. DI over Singleton applies DIP.
- **Section 5 (Structural Patterns):** Adapter often wraps objects created by a factory. Decorator chains are commonly assembled using a Builder-like approach.
- **Section 6 (Behavioral Patterns):** Strategy objects are frequently created by Factory Methods. Command objects are often constructed with Builders when they carry complex payloads.
- **Section 7 (Pattern Selection):** The signal-to-pattern mapping table includes all four creational patterns with their recognition signals.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn